```
import os, platform, subprocess, difflib, re, threading
import tkinter as tk
from tkinter import ttk, filedialog, messagebox
from datetime import datetime

# ── DPI Fix Windows 11 ──────────────────────────────────────────────
if platform.system() == "Windows":
    try:
        from ctypes import windll
        windll.shcore.SetProcessDpiAwareness(2)
    except Exception:
        try: windll.user32.SetProcessDPIAware()
        except: pass

try:
    import requests
    REQUESTS_OK = True
except ImportError:
    REQUESTS_OK = False

# ═══════════════════════════════════════════════════════════════════════
# ═══ CONFIG RÉSEAU
# ═══════════════════════════════════════════════════════════════════════
systeme   = platform.system()
PING_CMD  = "ping -n 4 " if systeme == "Windows" else "ping -c 4 "
ENCODAGE  = "cp850"      if systeme == "Windows" else "utf-8"

BATIMENTS = {
    "Hotel du Departement": 10, "Tour Jean Moulin": 11, "Mounier": 12,
    "Cheiron": 13, "Audibergue": 14, "Ginesy": 15, "Mont des
Merveilles": 16,
    "Esterel": 17, "Garage": 18, "Authion": 19, "Ferion": 21,
    "Ariane": 30, "Centaure": 31, "Mediatheque": 34,
}
BAT_LOWER     = {k.lower(): k for k in BATIMENTS}
IPS_ROUTAGE   = ["10.1.20.232"]
IPS_SNMP      = ["10.1.20.40"]
URLS_INTERNET = ["https://www.tf1.fr/"]
```

```
URLS_INTRANET = ["https://intranet/"]

#
================================================================================
=======
#  THÈMES
#
================================================================================
=======
THEMES = {
    "dark": {
        "BG":       "#0a0a0f",
        "S1":       "#111118",
        "S2":       "#18181f",
        "S3":       "#1f1f28",
        "S3H":      "#252530",
        "BORDER":   "#2a2a35",
        "ACCENT":   "#00d4aa",
        "ACCENT_D": "#00a884",
        "OK":       "#00d4aa",
        "KO":       "#ff4d6d",
        "WARN":     "#f5a623",
        "TEXT":     "#f0f0f5",
        "SUB":      "#8888a0",
        "DIM":      "#3a3a4a",
        "ICON":     "☀",
        "NEXT":     "Mode clair",
    },
    "light": {
        "BG":       "#f2f2f7",
        "S1":       "#ffffff",
        "S2":       "#f7f7fb",
        "S3":       "#ebebf2",
        "S3H":      "#e0e0ea",
        "BORDER":   "#d4d4e0",
        "ACCENT":   "#0099aa",
        "ACCENT_D": "#007a88",
        "OK":       "#0099aa",
        "KO":       "#e03050",
        "WARN":     "#b86800",
        "TEXT":     "#0f0f18",
        "SUB":      "#666680",
        "DIM":      "#c0c0cc",
```

```
        "ICON":      "☾",
        "NEXT":      "Mode sombre",
    },
}

T = dict(THEMES["dark"])    # thème actif (global mutable)

#
=================================================================================
#  LOGIQUE MÉTIER
#
=================================================================================
def est_ip_valide(ip):
    return bool(re.match(

r"^(25[0-5]|2[0-4]\d|[01]?\d\d?)(\.(25[0-5]|2[0-4]\d|[01]?\d\d?)){3}$",
ip))

def generer_ips_vlan(num):
    ips = [f"10.{num}.{num}.1", f"10.{200+num}.{num}.1"]
    for v in [4, 180, 181, 182, 186]:
        ips.append(f"10.{v}.{num}.1")
    return ips

def ping_ip(ip):
    if not est_ip_valide(ip):
        return False, "IP invalide"
    try:
        res = subprocess.run(PING_CMD + ip, shell=True,
                             stdout=subprocess.PIPE,
stderr=subprocess.PIPE, timeout=12)
        out = res.stdout.decode(ENCODAGE, errors="ignore") + \
              res.stderr.decode(ENCODAGE, errors="ignore")
        return ("TTL=" in out or "ttl=" in out), out.strip()
    except subprocess.TimeoutExpired:
        return False, "Timeout"
    except Exception as e:
        return False, str(e)

def test_url(url):
    if not REQUESTS_OK:
```

```
        return False, "requests non installé"
    try:
        r = requests.get(url, headers={"User-Agent": "Mozilla/5.0"},
                         verify=False, timeout=5)
        return r.status_code == 200, f"HTTP {r.status_code}"
    except requests.exceptions.SSLError:        return False, "Erreur
SSL"
    except requests.exceptions.ConnectionError: return False,
"Connexion refusée"
    except requests.exceptions.Timeout:         return False, "Timeout"
    except Exception as e:                      return False, str(e)

#
================================================================================
#  RAPPORT HTML
#
================================================================================
def generer_rapport_html(bat, num, resultats, ts, log_path):
    total = len(resultats)
    ok    = sum(1 for r in resultats if r["ok"])
    pct   = int(ok / total * 100) if total else 0
    rows  = "".join(
        f'<tr class="{"ok" if r["ok"] else "ko"}">'
        f'<td><span class="pill {"ok" if r["ok"] else "ko"}">{"OK" if
r["ok"] else "KO"}</span></td>'
        f'<td
class="dim">{r["type"]}</td><td><code>{r["cible"]}</code></td>'
        f'<td class="dim">{r["detail"]}</td></tr>'
        for r in resultats
    )
    return f"""<!DOCTYPE html><html lang="fr"><head><meta
charset="UTF-8">
<title>Rapport Switch — {bat}</title>
<style>
@import
url('https://fonts.googleapis.com/css2?family=DM+Sans:wght@400;500;600&
family=DM+Mono:wght@400;500&display=swap');
*{{box-sizing:border-box;margin:0;padding:0}}
body{{background:#0a0a0f;color:#f0f0f5;font-family:'DM
Sans',sans-serif;min-height:100vh;padding:48px 40px}}
.wrap{{max-width:960px;margin:0 auto}}
```

```
header{{display:flex;justify-content:space-between;align-items:center;m
argin-bottom:40px;padding-bottom:24px;border-bottom:1px solid #2a2a35}}
h1{{font-size:1.4rem;font-weight:600;letter-spacing:-.3px}}
.chip{{background:#18181f;border:1px solid #2a2a35;padding:5px
14px;border-radius:6px;font-size:.72rem;color:#8888a0;font-family:'DM
Mono',monospace}}
.meta{{color:#8888a0;font-family:'DM
Mono',monospace;font-size:.75rem;margin-top:5px}}
.kpis{{display:grid;grid-template-columns:repeat(4,1fr);gap:12px;margin
-bottom:20px}}
.kpi{{background:#111118;border:1px solid
#2a2a35;border-radius:10px;padding:22px 20px}}
.kpi
.v{{font-size:2rem;font-weight:600;line-height:1;margin-bottom:5px}}
.kpi
.l{{font-size:.65rem;color:#8888a0;text-transform:uppercase;letter-spac
ing:1.5px}}
.kpi.a .v{{color:#00d4aa}}.kpi.g .v{{color:#00d4aa}}.kpi.r
.v{{color:#ff4d6d}}.kpi.w .v{{color:#f5a623}}
.prog-wrap{{background:#111118;border:1px solid
#2a2a35;border-radius:10px;padding:20px 24px;margin-bottom:20px}}
.prog-top{{display:flex;justify-content:space-between;font-size:.75rem;
color:#8888a0;margin-bottom:10px}}
.bar{{background:#1f1f28;border-radius:4px;height:4px;overflow:hidden}}
.fill{{height:100%;border-radius:4px;background:#00d4aa;width:{pct}%}}
.table-card{{background:#111118;border:1px solid
#2a2a35;border-radius:10px;overflow:hidden}}
.table-head{{padding:14px
20px;font-size:.8rem;font-weight:600;border-bottom:1px solid
#2a2a35;display:flex;justify-content:space-between;align-items:center}}
.count{{font-size:.7rem;color:#8888a0;font-family:'DM Mono',monospace}}
table{{width:100%;border-collapse:collapse}}
th{{padding:9px
20px;font-size:.62rem;text-transform:uppercase;letter-spacing:1.5px;col
or:#8888a0;text-align:left;border-bottom:1px solid #1f1f28}}
td{{padding:11px 20px;font-size:.82rem;border-bottom:1px solid
#151519;vertical-align:middle}}
tr:last-child td{{border:none}}
.pill{{padding:2px
9px;border-radius:4px;font-size:.68rem;font-family:'DM
Mono',monospace;font-weight:500}}
.pill.ok{{background:rgba(0,212,170,.1);color:#00d4aa}}
.pill.ko{{background:rgba(255,77,109,.1);color:#ff4d6d}}
```

```
code{{background:#1f1f28;padding:1px
7px;border-radius:4px;font-family:'DM
Mono',monospace;font-size:.78rem;color:#a0a0c0}}
.dim{{color:#8888a0;font-size:.8rem}}
footer{{margin-top:32px;text-align:center;font-size:.7rem;color:#3a3a4a
;font-family:'DM Mono',monospace}}
</style></head><body><div class="wrap">
<header>
  <div><h1>Switch Test Report</h1><div class="meta">{bat}  · 
N°{num}  ·  {ts}</div></div>
  <span class="chip">CADAM Infrastructure</span>
</header>
<div class="kpis">
  <div class="kpi a"><div class="v">{total}</div><div class="l">Tests
total</div></div>
  <div class="kpi g"><div class="v">{ok}</div><div
class="l">Succès</div></div>
  <div class="kpi r"><div class="v">{total-ok}</div><div
class="l">Échecs</div></div>
  <div class="kpi w"><div class="v">{pct}%</div><div
class="l">Score</div></div>
</div>
<div class="prog-wrap">
  <div class="prog-top"><span>Disponibilité
réseau</span><span>{ok}/{total} réussis</span></div>
  <div class="bar"><div class="fill"></div></div>
</div>
<div class="table-card">
  <div class="table-head">Détail <span class="count">{total}
entrées</span></div>

<table><thead><tr><th>Statut</th><th>Type</th><th>Cible</th><th>Info</t
h></tr></thead>
  <tbody>{rows}</tbody></table>
</div>
<footer>Généré le {ts} · Tests_Switch v3.1 ·
{os.path.basename(log_path)}</footer>
</div></body></html>"""

#
==========================================================================
=======
#  DROPDOWN CUSTOM
```

```
#
==========================================================================
=======
class CustomDropdown(tk.Frame):
    def __init__(self, parent, values, textvariable, on_select=None,
**kw):
        super().__init__(parent, bg=T["S3"],
                         highlightthickness=1,
highlightbackground=T["BORDER"],
                         highlightcolor=T["ACCENT"])
        self._values   = values
        self._var      = textvariable
        self._callback = on_select
        self._open     = False
        self._popup    = None

        self._row = tk.Frame(self, bg=T["S3"])
        self._row.pack(fill="x")

        self._lbl = tk.Label(self._row, textvariable=self._var,
                             bg=T["S3"], fg=T["SUB"],
                             font=("Segoe UI", 9), anchor="w",
                             padx=12, pady=9, cursor="hand2")
        self._lbl.pack(side="left", fill="x", expand=True)

        self._cv = tk.Canvas(self._row, width=28, height=34,
                             bg=T["S3"], highlightthickness=0)
        self._cv.pack(side="right")
        self._draw_arrow(False)

        for w in (self, self._row, self._lbl, self._cv):
            w.bind("<Button-1>", self._toggle)
            w.bind("<Enter>",    lambda _: self._hover(True))
            w.bind("<Leave>",    lambda _: self._hover(False))

    def apply_theme(self):
        self.config(bg=T["S3"], highlightbackground=T["BORDER"])
        self._row.config(bg=T["S3"])
        self._lbl.config(bg=T["S3"], fg=T["TEXT"] if self._var.get()
else T["SUB"])
        self._cv.config(bg=T["S3"])
        self._draw_arrow(self._open)
```

```
    def _draw_arrow(self, up=False):
        self._cv.delete("all")
        cx = 14
        pts = [cx-4, 20, cx, 14, cx+4, 20] if up else [cx-4, 14, cx,
20, cx+4, 14]
        self._cv.create_line(*pts, fill=T["SUB"], width=1.5,
                             capstyle="round", joinstyle="round")

    def _hover(self, on):
        bg = T["S3H"] if on else T["S3"]
        self.config(highlightbackground=T["ACCENT"] if on else
T["BORDER"])
        self._lbl.config(bg=bg)
        self._cv.config(bg=bg)

    def _toggle(self, _=None):
        if self._open: self._close()
        else:          self._open_popup()

    def _open_popup(self):
        self._open = True
        self._draw_arrow(True)
        self.config(highlightbackground=T["ACCENT"])
        self.update_idletasks()
        x = self.winfo_rootx()
        y = self.winfo_rooty() + self.winfo_height()
        w = self.winfo_width()

        self._popup = tk.Toplevel(self)
        self._popup.overrideredirect(True)
        self._popup.configure(bg=T["BORDER"])

        item_h  = 34
        visible = min(len(self._values), 9)
        pop_h   = visible * item_h + 2
        self._popup.geometry(f"{w}x{pop_h}+{x}+{y}")
        self._popup.lift()

        inner = tk.Frame(self._popup, bg=T["S2"])
        inner.pack(fill="both", expand=True, padx=1, pady=1)

        sb = tk.Scrollbar(inner, orient="vertical", bg=T["S3"],
```

```
                            troughcolor=T["S2"],
activebackground=T["DIM"],
                            relief="flat", width=6, bd=0)
        cv = tk.Canvas(inner, bg=T["S2"], highlightthickness=0,
yscrollcommand=sb.set)
        if len(self._values) > 9:
            sb.pack(side="right", fill="y")
        cv.pack(side="left", fill="both", expand=True)
        sb.config(command=cv.yview)

        lf = tk.Frame(cv, bg=T["S2"])
        cv.create_window((0, 0), window=lf, anchor="nw", width=w - 8)
        current = self._var.get()

        for val in self._values:
            sel = (val == current)
            bg0 = T["S3"] if sel else T["S2"]
            fg0 = T["ACCENT"] if sel else T["TEXT"]
            hbg = T["S3H"]

            item = tk.Frame(lf, bg=bg0, cursor="hand2")
            item.pack(fill="x")
            ind = tk.Frame(item, bg=T["ACCENT"] if sel else bg0,
width=3)
            ind.pack(side="left", fill="y")
            lbl = tk.Label(item, text=val, bg=bg0, fg=fg0,
                           font=("Segoe UI", 9), anchor="w",
                           padx=12, pady=8, cursor="hand2")
            lbl.pack(side="left", fill="x", expand=True)

            def _en(e, f=item, l=lbl, i=ind, s=sel, h=hbg):
                if not s: f.config(bg=h); l.config(bg=h);
i.config(bg=h)
            def _lv(e, f=item, l=lbl, i=ind, s=sel, b=bg0):
                f.config(bg=b); l.config(bg=b); i.config(bg=b)
            def _cl(e, v=val): self._select(v)

            for w_ in (item, lbl):
                w_.bind("<Enter>",    _en)
                w_.bind("<Leave>",    _lv)
                w_.bind("<Button-1>", _cl)

        def _scroll(e):
```

```
            cv.yview_scroll(-1 * (e.delta // 120), "units"); return
"break"

        def _bind_rec(widget):
            widget.bind("<MouseWheel>", _scroll)
            for child in widget.winfo_children(): _bind_rec(child)

        _bind_rec(self._popup)
        lf.update_idletasks()
        cv.config(scrollregion=cv.bbox("all"))
        self._popup.bind("<FocusOut>", lambda _: self._close())
        self._popup.focus_set()
        self._popup.bind("<Escape>", lambda _: self._close())

    def _select(self, value):
        self._var.set(value)
        self._lbl.config(fg=T["TEXT"])
        self._close()
        if self._callback: self._callback()

    def _close(self):
        self._open = False
        self._draw_arrow(False)
        self.config(highlightbackground=T["BORDER"])
        if self._popup:
            try:    self._popup.destroy()
            except: pass
            self._popup = None

    def get(self): return self._var.get()


#
=================================================================================
=======
#  APPLICATION
#
=================================================================================
=======
class App(tk.Tk):
    def __init__(self):
        super().__init__()
        self.title("Switch Tests — CADAM")
```

```
        self.configure(bg=T["BG"])
        self.geometry("1060x700")
        self.minsize(860, 580)

        self.resultats    = []
        self.log_lines    = []
        self.running      = False
        self.num_batiment = None
        self._log_path    = ""
        self.bat_var      = tk.StringVar()
        self.log_var      = tk.StringVar()
        self.prog_var     = tk.DoubleVar()
        self._mode        = "dark"

        # Registres pour le theming
        self._bg_reg  = []   # (widget, couleur_key)
        self._fg_reg  = []   # (widget, fg_key, bg_key)
        self._sep_reg = []   # séparateurs
        self._hl_reg  = []   # (widget, hl_key)

        self._ttk_style = ttk.Style(self)
        try:    self._ttk_style.theme_use("clam")
        except: pass
        self._refresh_ttk()
        self._build()

    # ── Build ──────────────────────────────────────────────────────────────
    def _build(self):
        # ── Sidebar ──────────────────────────────────────────────────────
        self.sb = tk.Frame(self, bg=T["S1"], width=272)
        self.sb.pack(side="left", fill="y")
        self.sb.pack_propagate(False)
        self._bg(self.sb, "S1")

        hdr = self._frm(self.sb, "S1")
        hdr.pack(fill="x", padx=28, pady=(28, 0))

        dot_row = self._frm(hdr, "S1")
        dot_row.pack(anchor="w", pady=(0, 10))
        self._lbl_mk(dot_row, "●",                    "S1", "ACCENT",
("Segoe UI", 7)).pack(side="left")
```

```
        self._lbl_mk(dot_row, "  CADAM Infrastructure", "S1", "SUB",
("Segoe UI", 7)).pack(side="left")

        self._lbl_mk(hdr, "Switch\nTests", "S1", "TEXT",
                     ("Segoe UI", 18, "bold"),
justify="left").pack(anchor="w")
        self._lbl_mk(hdr, "Outil de diagnostic réseau", "S1", "SUB",
                     ("Segoe UI", 8)).pack(anchor="w", pady=(4, 0))

        self._sep_mk(self.sb, padx=28, pady=(20, 0))

        frm = self._frm(self.sb, "S1", padx=28)
        frm.pack(fill="x", pady=16)

        self._lbl_mk(frm, "BÂTIMENT", "S1", "SUB", ("Segoe UI", 7,
"bold")).pack(anchor="w")
        self.bat_cb = CustomDropdown(frm,
values=sorted(BATIMENTS.keys()),
                                     textvariable=self.bat_var,
on_select=self._on_bat)
        self.bat_cb.pack(fill="x", pady=(4, 14))

        self._lbl_mk(frm, "FICHIER LOG  (OPTIONNEL)", "S1", "SUB",
                     ("Segoe UI", 7, "bold")).pack(anchor="w")
        self._entry_mk(frm, self.log_var)

        spacer1 = tk.Frame(self.sb, bg=T["S1"])
        spacer1.pack(pady=4)
        self._bg(spacer1, "S1")
        btn_wrap = self._frm(self.sb, "S1", padx=28)
        btn_wrap.pack(fill="x")
        self.btn_run = self._btn_accent(btn_wrap, "Lancer les tests",
self._lancer)

        self._sep_mk(self.sb, padx=28, pady=(20, 0))

        kpi_outer = self._frm(self.sb, "S1", padx=28)
        kpi_outer.pack(fill="x", pady=16)
        grid = self._frm(kpi_outer, "S1")
        grid.pack(fill="x")
        grid.columnconfigure(0, weight=1)
        grid.columnconfigure(1, weight=1)
```

```
        self.kpi_tot   = self._kpi_mk(grid, "TOTAL",  "ACCENT", 0, 0)
        self.kpi_ok    = self._kpi_mk(grid, "SUCCÈS", "OK",     0, 1)
        self.kpi_ko    = self._kpi_mk(grid, "ÉCHECS", "KO",     1, 0)
        self.kpi_score = self._kpi_mk(grid, "SCORE",  "WARN",   1, 1)
        self._kpi_color_keys = ["ACCENT", "OK", "KO", "WARN"]

        prog_f = self._frm(self.sb, "S1", padx=28)
        prog_f.pack(fill="x", pady=(0, 4))
        self.prog_bar = ttk.Progressbar(prog_f, variable=self.prog_var,

style="Pro.Horizontal.TProgressbar", maximum=100)
        self.prog_bar.pack(fill="x")
        self.prog_lbl = self._lbl_mk(prog_f, "", "S1", "DIM", ("Segoe
UI", 7))
        self.prog_lbl.pack(anchor="e", pady=(3, 0))

        spacer2 = tk.Frame(self.sb, bg=T["S1"])
        spacer2.pack(fill="both", expand=True)
        self._bg(spacer2, "S1")
        self._sep_mk(self.sb, padx=28)

        act = self._frm(self.sb, "S1", padx=28, pady=16)
        act.pack(fill="x")
        self.btn_html  = self._btn_ghost(act, "Exporter HTML",
self._export_html)
        self.btn_clear = self._btn_ghost(act, "Effacer",
self._clear, dim=True)

        # ── Bouton toggle thème
──────────────────────────────────────────────────

        self._sep_mk(self.sb, padx=28)
        theme_row = self._frm(self.sb, "S1", padx=28, pady=14)
        theme_row.pack(fill="x")

        # Toggle switch visuel
        self.toggle_cv = tk.Canvas(theme_row, width=36, height=20,
                                   bg=T["S1"], highlightthickness=0,
cursor="hand2")
        self.toggle_cv.pack(side="left")
        self._bg(self.toggle_cv, "S1")
        self._draw_toggle()
```

```
        self.theme_lbl = self._lbl_mk(theme_row, T["NEXT"], "S1",
"SUB",
                                      ("Segoe UI", 8))
        self.theme_lbl.pack(side="left", padx=(8, 0))

        self.toggle_cv.bind("<Button-1>", lambda _:
self._toggle_theme())
        self.theme_lbl.bind("<Button-1>", lambda _:
self._toggle_theme())

        self._lbl_mk(self.sb, "v3.1", "S1", "DIM",
                     ("Segoe UI", 7)).pack(side="bottom", pady=(0, 12))

        # ── Zone console
──────────────────────────────────────────────────────────
        self.main_frame = tk.Frame(self, bg=T["BG"])
        self.main_frame.pack(side="left", fill="both", expand=True)
        self._bg(self.main_frame, "BG")

        top = self._frm(self.main_frame, "BG", padx=30, pady=20)
        top.pack(fill="x")

        self._lbl_mk(top, "Console", "BG", "TEXT",
                     ("Segoe UI", 11, "bold")).pack(side="left")

        rg = self._frm(top, "BG")
        rg.pack(side="right")
        self.status_dot = self._lbl_mk(rg, "●", "BG", "DIM", ("Segoe
UI", 8))
        self.status_dot.pack(side="left")
        self.status_lbl = self._lbl_mk(rg, "  En attente", "BG", "SUB",
("Segoe UI", 8))
        self.status_lbl.pack(side="left")

        self._console_sep = tk.Frame(self.main_frame, bg=T["BORDER"],
height=1)
        self._console_sep.pack(fill="x", padx=30)
        self._sep_reg.append(self._console_sep)

        wrap = self._frm(self.main_frame, "BG", padx=30, pady=20)
        wrap.pack(fill="both", expand=True)

        self.console_frame = tk.Frame(wrap, bg=T["S2"],
```

```
                                        highlightthickness=1,
                                        highlightbackground=T["BORDER"])
        self.console_frame.pack(fill="both", expand=True)
        self._bg(self.console_frame, "S2")
        self._hl_reg.append((self.console_frame, "BORDER"))

        self.console_sb = tk.Scrollbar(self.console_frame, bg=T["S3"],
                                        troughcolor=T["S2"],
activebackground=T["DIM"],
                                        relief="flat", width=8, bd=0)
        self.console_sb.pack(side="right", fill="y", padx=(0, 2),
pady=2)

        self.console = tk.Text(
            self.console_frame,
            bg=T["S2"], fg=T["TEXT"],
            insertbackground=T["ACCENT"],
            font=("Consolas", 9), relief="flat", bd=0,
            padx=20, pady=16, state="disabled",
            spacing1=1, spacing2=0, spacing3=4,
            wrap="word", yscrollcommand=self.console_sb.set,
            cursor="arrow",
            selectbackground=T["S3"], selectforeground=T["TEXT"],
        )
        self.console.pack(side="left", fill="both", expand=True)
        self.console_sb.config(command=self.console.yview)
        self._refresh_tags()
        self._cw("  Prêt. Sélectionnez un bâtiment et lancez les
tests.\n\n", "sub")

    # ── Helpers création widgets
──────────────────────────────────────────────────

    def _frm(self, parent, bg_key, padx=0, pady=0):
        f = tk.Frame(parent, bg=T[bg_key], padx=padx, pady=pady)
        self._bg(f, bg_key)
        return f

    def _bg(self, w, key):
        self._bg_reg.append((w, key))

    def _lbl_mk(self, parent, text, bg_key, fg_key, font, **kw):
        lbl = tk.Label(parent, text=text, bg=T[bg_key], fg=T[fg_key],
font=font, **kw)
```

```
        self._fg_reg.append((lbl, fg_key, bg_key))
        return lbl

    def _sep_mk(self, parent, padx=0, pady=(0, 0)):
        s = tk.Frame(parent, bg=T["BORDER"], height=1)
        s.pack(fill="x", padx=padx, pady=pady)
        self._sep_reg.append(s)

    def _entry_mk(self, parent, var):
        f = tk.Frame(parent, bg=T["S3"],
                     highlightthickness=1,
highlightbackground=T["BORDER"])
        f.pack(fill="x", pady=(4, 0))
        self._bg(f, "S3"); self._hl_reg.append((f, "BORDER"))
        e = tk.Entry(f, textvariable=var, bg=T["S3"], fg=T["TEXT"],
                     insertbackground=T["ACCENT"],
                     relief="flat", font=("Segoe UI", 9), bd=8,
highlightthickness=0)
        e.pack(fill="x")
        self._fg_reg.append((e, "TEXT", "S3"))
        e.bind("<FocusIn>",  lambda _:
f.config(highlightbackground=T["ACCENT"]))
        e.bind("<FocusOut>", lambda _:
f.config(highlightbackground=T["BORDER"]))

    def _btn_accent(self, parent, text, cmd):
        b = tk.Label(parent, text=text, bg=T["ACCENT"], fg=T["BG"],
                     font=("Segoe UI", 9, "bold"), pady=11,
cursor="hand2", anchor="center")
        b.pack(fill="x")
        b.bind("<Enter>",    lambda _: b.config(bg=T["ACCENT_D"]))
        b.bind("<Leave>",    lambda _: b.config(bg=T["ACCENT"]))
        b.bind("<Button-1>", lambda _: cmd())
        return b

    def _btn_ghost(self, parent, text, cmd, dim=False):
        key = "DIM" if dim else "SUB"
        b = tk.Label(parent, text=text, bg=T["S1"], fg=T[key],
                     font=("Segoe UI", 8), pady=5, cursor="hand2",
anchor="w")
        b.pack(fill="x")
        self._fg_reg.append((b, key, "S1"))
        b.bind("<Enter>",    lambda _: b.config(fg=T["TEXT"]))
```

```
        b.bind("<Leave>",    lambda _: b.config(fg=T[key]))
        b.bind("<Button-1>", lambda _: cmd())
        return b

    def _kpi_mk(self, parent, label, color_key, row, col):
        pad_r = 6 if col == 0 else 0
        pad_b = 6 if row == 0 else 0
        f = tk.Frame(parent, bg=T["S3"],
                     highlightthickness=1,
highlightbackground=T["BORDER"])
        f.grid(row=row, column=col, sticky="nsew", padx=(0, pad_r),
pady=(0, pad_b))
        self._bg(f, "S3"); self._hl_reg.append((f, "BORDER"))
        lbl = tk.Label(f, text=label, bg=T["S3"], fg=T["DIM"],
font=("Segoe UI", 6, "bold"))
        lbl.pack(pady=(10, 0))
        self._fg_reg.append((lbl, "DIM", "S3"))
        val = tk.Label(f, text="—", bg=T["S3"], fg=T[color_key],
font=("Segoe UI", 17, "bold"))
        val.pack(pady=(2, 10))
        self._fg_reg.append((val, color_key, "S3"))
        return val

    # ── Toggle switch dessiné
──────────────────────────────────────────────────

    def _draw_toggle(self):
        self.toggle_cv.delete("all")
        is_light = (self._mode == "light")
        # Piste
        track_col = T["ACCENT"] if is_light else T["BORDER"]
        self.toggle_cv.create_rounded_rect = lambda *a, **kw: None
        self.toggle_cv.create_oval(0, 0, 36, 20, fill=track_col,
outline="")
        # Bouton
        x = 18 if is_light else 2
        self.toggle_cv.create_oval(x, 2, x + 16, 18, fill=T["S1"],
outline="")

    # ── Thème
──────────────────────────────────────────────────────────────

    def _toggle_theme(self):
        global T
        self._mode = "light" if self._mode == "dark" else "dark"
```

```
        T = dict(THEMES[self._mode])
        self._apply_theme()

    def _apply_theme(self):
        self.configure(bg=T["BG"])

        for w, key in self._bg_reg:
            try: w.config(bg=T[key])
            except: pass

        for w, fg_key, bg_key in self._fg_reg:
            try: w.config(fg=T[fg_key], bg=T[bg_key])
            except: pass

        for s in self._sep_reg:
            try: s.config(bg=T["BORDER"])
            except: pass

        for w, key in self._hl_reg:
            try: w.config(highlightbackground=T[key])
            except: pass

        # Bouton run
        self.btn_run.config(bg=T["ACCENT"], fg=T["BG"])

        # Toggle
        self.toggle_cv.config(bg=T["S1"])
        self._draw_toggle()
        self.theme_lbl.config(text=T["NEXT"], bg=T["S1"], fg=T["SUB"])

        # Console
        self.console_frame.config(bg=T["S2"])
        self.console_sb.config(bg=T["S3"], troughcolor=T["S2"],
activebackground=T["DIM"])
        self.console.config(bg=T["S2"], fg=T["TEXT"],
                            insertbackground=T["ACCENT"],
                            selectbackground=T["S3"],
selectforeground=T["TEXT"])
        self._refresh_tags()

        # KPI valeurs (couleurs dynamiques)
        kpi_widgets = [self.kpi_tot, self.kpi_ok, self.kpi_ko,
self.kpi_score]
```

```
        for w, ck in zip(kpi_widgets, self._kpi_color_keys):
            try: w.config(fg=T[ck])
            except: pass

        # Status (garder couleur courante si running)
        if not self.running:
            self.status_dot.config(fg=T["DIM"])
            self.status_lbl.config(fg=T["SUB"])

        # Dropdown
        self.bat_cb.apply_theme()

        # TTK
        self._refresh_ttk()

    def _refresh_ttk(self):
        self._ttk_style.configure("Pro.Horizontal.TProgressbar",
                                  troughcolor=T["S3"],
background=T["ACCENT"],
                                  bordercolor=T["S3"],
lightcolor=T["ACCENT"],
                                  darkcolor=T["ACCENT_D"], thickness=3)

    def _refresh_tags(self):
        self.console.tag_config("ok",   foreground=T["OK"],
font=("Consolas", 9, "bold"))
        self.console.tag_config("ko",   foreground=T["KO"],
font=("Consolas", 9, "bold"))
        self.console.tag_config("warn", foreground=T["WARN"])
        self.console.tag_config("info", foreground=T["ACCENT"])
        self.console.tag_config("sub",  foreground=T["SUB"])
        self.console.tag_config("dim",  foreground=T["DIM"])
        self.console.tag_config("head", foreground=T["TEXT"],
font=("Consolas", 9, "bold"))
        self.console.tag_config("ip",   foreground=T["SUB"],
font=("Consolas", 9))
        self.console.tag_config("type", foreground=T["ACCENT"],
font=("Consolas", 9))

    # ── Logique tests
──────────────────────────────────────────────────────────

    def _on_bat(self, _=None):
        nom = self.bat_var.get().strip().lower()
```

```
        if nom in BAT_LOWER:
            self.num_batiment = BATIMENTS[BAT_LOWER[nom]]
        else:
            m = difflib.get_close_matches(nom, BAT_LOWER.keys(), n=1,
cutoff=0.55)
            if m:
                correct = BAT_LOWER[m[0]]
                self.bat_var.set(correct)
                self.num_batiment = BATIMENTS[correct]
            else:
                self.num_batiment = None

    def _lancer(self):
        if self.running: return
        if not self.bat_var.get().strip() or self.num_batiment is None:
            messagebox.showerror("Erreur", "Veuillez sélectionner un
bâtiment.")
            return
        self.running = True
        self.btn_run.config(text="En cours…", bg=T["S3"], fg=T["SUB"])
        self.resultats, self.log_lines = [], []
        self.prog_var.set(0)
        self._upd_kpis()
        self._set_status("En cours", T["WARN"])
        threading.Thread(target=self._run, daemon=True).start()

    def _run(self):
        ts   = datetime.now().strftime("%Y%m%d-%H%M%S")
        bat  = self.bat_var.get().strip()
        num  = self.num_batiment
        log  = self.log_var.get().strip()
        path = f"{log}.log" if log else f"resultats_{bat.replace('
','_')}_{ts}.log"
        self._log_path = path

        tasks = (
            [("VLAN",     "ping", ip) for ip in generer_ips_vlan(num)]
+
            [("Routage",  "ping", ip) for ip in IPS_ROUTAGE] +
            [("SNMP",     "ping", ip) for ip in IPS_SNMP]    +
            [("Internet", "url",  u)  for u  in URLS_INTERNET] +
            [("Intranet", "url",  u)  for u  in URLS_INTRANET]
        )
```

```
        total = len(tasks)

        ts_nice = datetime.now().strftime("%d/%m/%Y  %H:%M:%S")
        self._cw(f"\n  {bat}  ·  N°{num}\n", "head")
        self._cw(f"  {ts_nice}\n", "sub")
        self._cw(f"  {'—'*46}\n\n", "dim")

        for idx, (type_, proto, cible) in enumerate(tasks):
            self._cw(f"  ", "dim")
            self._cw(f"{type_:<9}", "type")
            self._cw(f"  {cible}\n", "ip")
            ok, detail = ping_ip(cible) if proto == "ping" else
test_url(cible)
            info = "Accessible" if ok else detail[:56]
            self._cw(f"  {'✓' if ok else '✗'}  {info}\n\n", "ok" if ok
else "ko")
            self.resultats.append({"ok": ok, "type": type_, "cible":
cible, "detail": info})
            self.log_lines.append(f"[{type_}] {cible} → {'OK' if ok
else 'KO'} — {info}\n")
            pct = int((idx + 1) / total * 100)
            self.after(0, self._upd_prog, pct, idx + 1, total)
            self.after(0, self._upd_kpis)

        try:
            with open(path, "w", encoding="utf-8") as f:
                f.write(f"Tests Switch CADAM — {bat} (N°{num}) —
{ts}\n\n")
                f.writelines(self.log_lines)
            self._cw(f"  {'—'*46}\n", "dim")
            self._cw(f"  Log  →  {path}\n", "sub")
        except Exception as e:
            self._cw(f"  ⚠  Log impossible : {e}\n", "warn")

        ok_n = sum(1 for r in self.resultats if r["ok"])
        verdict = "Tous les tests réussis" if ok_n == total else
f"{ok_n}/{total} tests réussis"
        self._cw(f"  {verdict}\n\n", "ok" if ok_n == total else "warn")
        self.after(0, self._done)

    def _done(self):
        self.running = False
```

```
        self.btn_run.config(text="Lancer les tests", bg=T["ACCENT"],
fg=T["BG"])
        ok  = sum(1 for r in self.resultats if r["ok"])
        tot = len(self.resultats)
        self._set_status(f"Terminé  ·  {ok}/{tot} OK", T["OK"] if ok ==
tot else T["WARN"])

    def _set_status(self, text, color):
        self.status_lbl.config(text=f"  {text}", fg=color)
        self.status_dot.config(fg=color)

    def _cw(self, text, tag=""):
        def _w():
            self.console.config(state="normal")
            self.console.insert("end", text, tag) if tag else
self.console.insert("end", text)
            self.console.see("end")
            self.console.config(state="disabled")
        self.after(0, _w)

    def _upd_prog(self, pct, done, total):
        self.prog_var.set(pct)
        self.prog_lbl.config(text=f"{done} / {total}  ·  {pct}%")

    def _upd_kpis(self):
        tot = len(self.resultats)
        ok  = sum(1 for r in self.resultats if r["ok"])
        pct = f"{int(ok/tot*100)}%" if tot else "—"
        self.kpi_tot.config(text=str(tot) if tot else "—")
        self.kpi_ok.config(text=str(ok)   if tot else "—")
        self.kpi_ko.config(text=str(tot - ok) if tot else "—")
        self.kpi_score.config(text=pct)

    def _export_html(self):
        if not self.resultats: return
        ts   = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
        bat  = self.bat_var.get().strip()
        html = generer_rapport_html(bat, self.num_batiment,
self.resultats, ts, self._log_path)
        path = filedialog.asksaveasfilename(
            defaultextension=".html", filetypes=[("HTML", "*.html")],
            initialfile=f"rapport_{bat.replace(' ','_')}.html",
            title="Enregistrer le rapport HTML"
```

```
        )
        if path:
            try:
                with open(path, "w", encoding="utf-8") as f:
f.write(html)
                import webbrowser
                webbrowser.open(f"file://{os.path.abspath(path)}")
            except Exception as e:
                messagebox.showerror("Erreur", str(e))

    def _clear(self):
        self.console.config(state="normal")
        self.console.delete("1.0", "end")
        self.console.config(state="disabled")
        self._cw("  Prêt. Sélectionnez un bâtiment et lancez les
tests.\n\n", "sub")
        self.resultats, self.log_lines = [], []
        self.prog_var.set(0)
        self.prog_lbl.config(text="")
        self._upd_kpis()
        self._set_status("En attente", T["DIM"])

#
==========================================================================
=======
if __name__ == "__main__":
    app = App()
    app.mainloop()
```